

Miami-Dade College North Campus

Group 1

CTF Write-up 9: Enumeration & Exploitation (NCL)

Olushola Adebisi

Christopher Jones

Jennifer Lorenzo

Bryan Martin

Giancarlo Montes

Ernesto Reyes

Christopher Sanchez

David Spivey

CIS 4891

April 20th, 2024

Professor Nelly Delessy

Completed by: Bryan Martin

Lab name: Python 2 (Easy)

Method of application: Kali Linux, Python, uncompyle6, rot13

Procedure:

The downloaded file needs to be in a readable format. We use the following commands firstly to download the tool needed to turn the .pyc file into a readable file (.py) for us to understand the python code. With uncompyle6 we can see the requirements needed.

```
$ pip install uncompyle6

(drago@kali)-[~/Downloads]
$ vim PYTHON2.pyc

(drago@kali)-[~/Downloads]
$ python uncompyle6 PYTHON2.pyc
# uncompyle6 version 3.9.1
# Python bytecode version base 2.7 (62211)
# Decompiled from: Python 3.11.8 (main, Feb  7 2024, 21:52:08)
# Embedded file name: NCL-2015-Python2.py
# Compiled at: 2015-11-12 14:43:01
import sys

def main():
    if len(sys.argv) != 2:
        print 'Invalid args'
        return
    password = sys.argv[1]
    counter = 0
    vals = list('tfzbwlyzljylawhzzdvky')
    if len(password) != len(vals):
        print 'incorrect'
        return
    while counter < len(password):
        x = ord(password[counter]) + 7
        if x > ord('z'):
            x -= 26
        if chr(x) != vals[counter]:
            print 'incorrect'
            return
        counter += 1

    print 'correct'

if __name__ == '__main__':
    main()

# okay decompiling PYTHON2.pyc
```

The highlighted code gives us a hint that it's a letter substitution cipher that replaces a letter with the 7th letter. Also there are 26 letters in the alphabet. We plug it into

rot13.com using ROT19 since the word list has already moved down 7 letters in the alphabet. ROT19 will move the word list back up by 7 letters giving us the correct password.

rot13.com

[About ROT13](#)

tfzbwlyzljylawhzzdvyk



ROT19 ▼



mysupersecretpassword

```
$ python python2.py mysupersecretpassword  
correct
```

Flag proof: Answer: **mysupersecretpassword**

Python 2 (Easy) (50 points)

Cyber Command

What is an input to this program that will result in a correct validation? (50 pts)

7:19:40 pm

Bryan Martin

mysupersecretpassword

8:31:30 pm

Cyber Command

✓ Correct! Good job so far.

8:31:30 pm

Cyber Command

🚩 You are done. Thank you for completing this mission. [Onto the next challenge! →](#)

8:45:36 pm

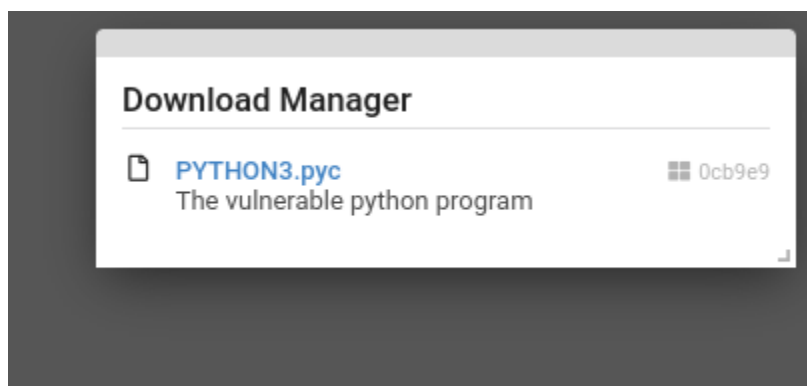
Completed by: Giancarlo Montes

Lab name: Python3 (Medium)

Method of application:

Procedure:

1. Download the file "PYTHON3.pyc"



2. Once downloaded we will decompile the file to human readable code:

Use command: “ **uncompyle6 PYTHON3.pyc**”

```

(root@kali)-[/home/kali/Downloads]
# uncompile6 PYTHON3.pyc
# uncompile6 version 3.9.1
# Python bytecode version base 2.7 (62211)
# Decompiled from: Python 3.11.6 (main, Oct 8 2023, 05:06:43) [GCC 13.2.0]
# Embedded file name: NCL-2015-Python3.py
# Compiled at: 2015-11-12 20:09:05
import sys

def main():
    if len(sys.argv) != 2:
        print 'Invalid args'
        return
    password = sys.argv[1]
    builder = 0
    for c in password:
        builder += ord(c)

    if __name__ == '__main__':
        main()

    builder = builder << 2
    builder = ~builder
    builder = builder ^ 12648430
    builder = ~builder
    if builder == 12645638 and ord(password[0]) == 78 and len(password) == 11:
        print 'correct'
    else:
        print 'incorrect'

if __name__ == '__main__':
    main()

# okay decompiling PYTHON3.pyc

```

- Next we convert the compiled pyc file to a standard python .py file with the following command:

Uncompile6 PYTHON3.pyc > Python3.py

```

(root@kali)-[/home/kali/Downloads]
# uncompile6 PYTHON3.pyc > Python3.py

```

- Next we will evaluate and edit the code a bit using: **“Vim Python3.py”**

We immediately get a notification of some sort of **SyntaxError** where we are missing parentheses

6. We fix our error easily by adding proper quotes and putting parentheses around any printable text:

```
# Compiled at: 2019-11-12 20:09:03
import sys

def main():
    if len(sys.argv) != 2:
        print("Invalid args")
        return
    password = sys.argv[1]
    builder = 0
    for c in password:
        builder += ord(c)

    builder = builder << 2
    builder = ~builder
    builder = builder ^ 12648430
    builder = ~builder
    if builder == 12645638 and ord(password[0]) == 78 and len(password) == 11:
        print("correct")
    else:
        print("incorrect")

if __name__ == '__main__':
    main()

# okay decompiling PYTHON3.pyc
~
~
```

Now we run our code with a test password and it works as intended:

```
(root@kali)-[/home/kali/Downloads]
# python Python3.py dfsjejed
incorrect
```

7. Now we will make our code more clean and easier to detect and figure out our password:

I begin by breaking down my code and creating if/else statements for each part of my password:

- builder** = specific pattern of password
- ord** = returns the number representing the unicode code of a specified character
- len** = the total length of the password
- all conditions met** = that all parts of password have been met and it is a valid password

```

File Actions Edit View Help
# uncompile6 version 3.9.1
# Python bytecode version base 2.7 (62211)
# Decompiled from: Python 3.11.6 (main, Oct 8 2023, 05:06:43) [GCC 13.2.0]
# Embedded file name: NCL-2015-Python3.py
# Compiled at: 2015-11-12 20:09:05
import sys

def main():
    if len(sys.argv) != 2:
        print("Invalid args")
        return
    password = sys.argv[1]
    builder = 0
    for c in password:
        builder += ord(c)
    builder = builder << 2
    builder = ~builder
    builder = builder ^ 12648430
    builder = ~builder

    if builder == 12645638:
        print("correct builder")
    else:
        print("wrong builder")

    if ord(password[0]) == 78:
        print("coorect ord")
    else:
        print("wrong ord")

    if len(password) == 11:
        print("correct length")
    else:
        print("incorrect length")

    if builder == 12645638 and ord(password[0]) == 78 and len(password) == 11:
        print("correct")
    else:
        print("Sorry you have guessed the incorrect password Please try again")

if __name__ == '__main__':
    main()

# okay decompiling python3.pyc
~

```

```

if builder == 12645638:
    print("correct builder")
else:
    print("wrong builder")

if ord(password[0]) == 78:
    print("coorect ord")
else:
    print("wrong ord")

if len(password) == 11:
    print("correct length")
else:
    print("incorrect length")

if builder == 12645638 and ord(password[0]) == 78 and len(password) == 11:
    print("correct")
else:
    print("Sorry you have guessed the incorrect password Please try again")

```

Here with these new statements we see in each field if the condition is met we will print that it is correct and if it is wrong we will print incorrect and the user must try again to find the password, and if the user gets 1 correct and 2 wrong it will let the user know, and if all conditions are met will print correct password overall.

```
(root@kali)-[/home/kali/python-re]
# python python3.py dfsjejed
wrong builder
wrong ord
incorrect length
Sorry you have guessed the incorrect password Please try again
```

As shown above, with the password tried now the code tells me which parts I got wrong and overall the password is incorrect and please try again.

8. Now that our code is clean lets break down each section:

OUR PASSWORD MUST CONTAIN:

A builder -> Password must equal (a vlaue of 12645638) in ascii table

A ord(password[0]) == 78 -> AKA: (The password's 1st character must equal a value of = 78 from an ascii carachter).

Len(password) == 1 -> The Length of the password must be equal to (11) characters (no more no less)

- Builder:

```
if builder == 12645638:
    print("correct builder")
else:
```

This tells us the total sum of our password will be equal to the ascii length of (12645638)

- Ord:

```
if ord(password[0]) == 78:
    print("coorect ord")
```

Here we see that the first character (0) in our password is = 78 on the ascii table:

After a quick google search we find Ascii (78) = N

ASCII Table

Dec = Decimal Value
Char = Character

'5' has the int value 53
if we write '5'-'0' it evaluates to 53-48, or the int 5
if we write char c = 'B'+32; then c stores 'b'

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL (null)	32	SPACE	64	@	96	`
1	SOH (start of heading)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	\$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	TAB (horizontal tab)	41	)	73	I	105	i
10	LF (NL line feed, new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (NP form feed, new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (device control 1)	49	1	81	Q	113	q
18	DC2 (device control 2)	50	2	82	R	114	r
19	DC3 (device control 3)	51	3	83	S	115	s
20	DC4 (device control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of trans. block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL

So now we know our Password will start with (N???????)

- Len:

```
if len(password) == 11:
    print("correct length")
```

Here we see our password will contain a Length of (11 characters)

We now know our password:

- will have an ascii total value of (12645638)
- Our first character in our password will be (N) -> with a value of 78 in ascii
- Our final clue is our password will be 11 characters long

So lets try to figure out a password:

- We know that our length is (11 and first character is N)
We got: (N??????????) so far
- Next we will find the remaining (10 characters) we can run the following JavaScript code to find the remaining 10 characters of ascii that will sum up to (12645638)

```
> ~(12645638)
-12645639
> ~(12645638) ^ 12648430
-2793
> ~(~(12645638) ^ 12648430)
2792
> ~(~(12645638) ^ 12648430) >> 2
698
> 698 - 78
620
> 620 / 10
62
```

-We get 62 (for our remaining 10 values)

60	<
61	=
62	>
63	?

-We search 62 on Ascii table and get (>)

-Now we can attempt our new Password: (N>>>>>>>>>)

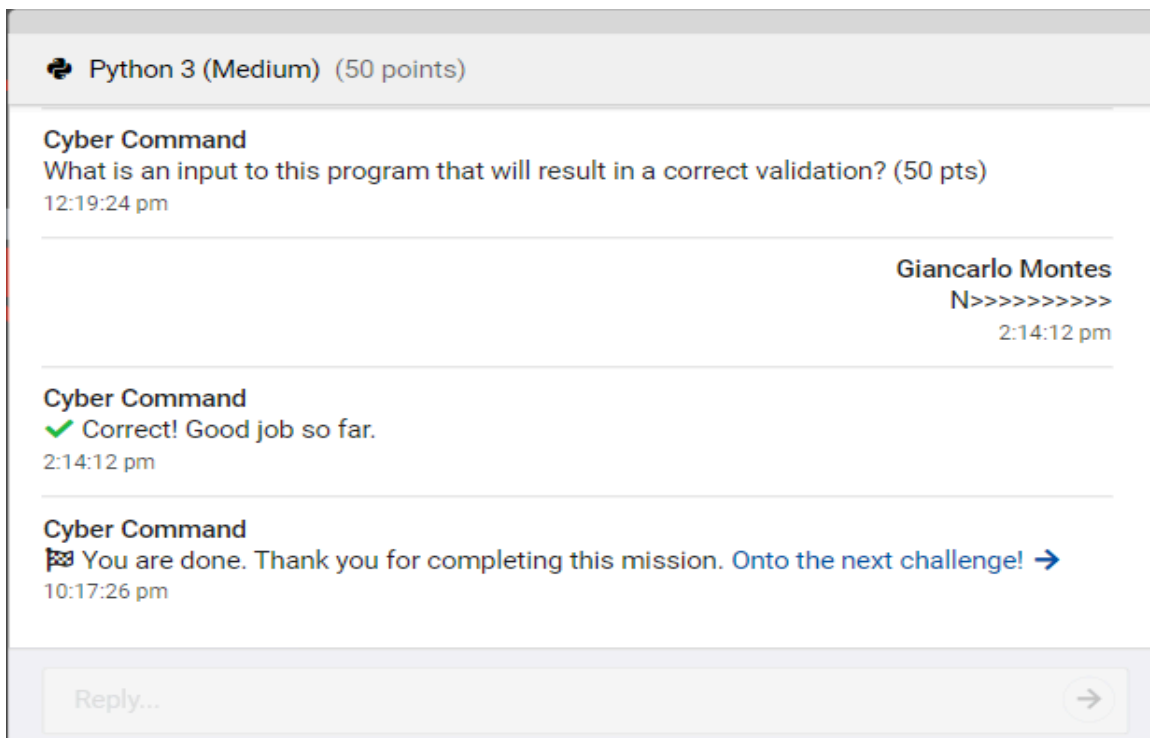
(because the password uses unique characters such as > we must put it in quotes so python can read it ' ')

```
(root@kali)-[/home/kali/python-re]
# python python3.py 'N>>>>>>>>>'
correct builder
coorect ord
correct length
correct
```

SUCCESS we guessed the right password!

(Note although this password worked if we find other ascii values that sum up to our build the password will also be valid)

Flag proof:



The screenshot shows a chat window titled "Python 3 (Medium) (50 points)". The chat history includes:

- Cyber Command**: "What is an input to this program that will result in a correct validation? (50 pts)" at 12:19:24 pm.
- Giancarlo Montes**: "N>>>>>>>>>" at 2:14:12 pm.
- Cyber Command**: "✓ Correct! Good job so far." at 2:14:12 pm.
- Cyber Command**: "🚩 You are done. Thank you for completing this mission. [Onto the next challenge!](#) ➔" at 10:17:26 pm.

At the bottom, there is a "Reply..." input field and a send button (➔).

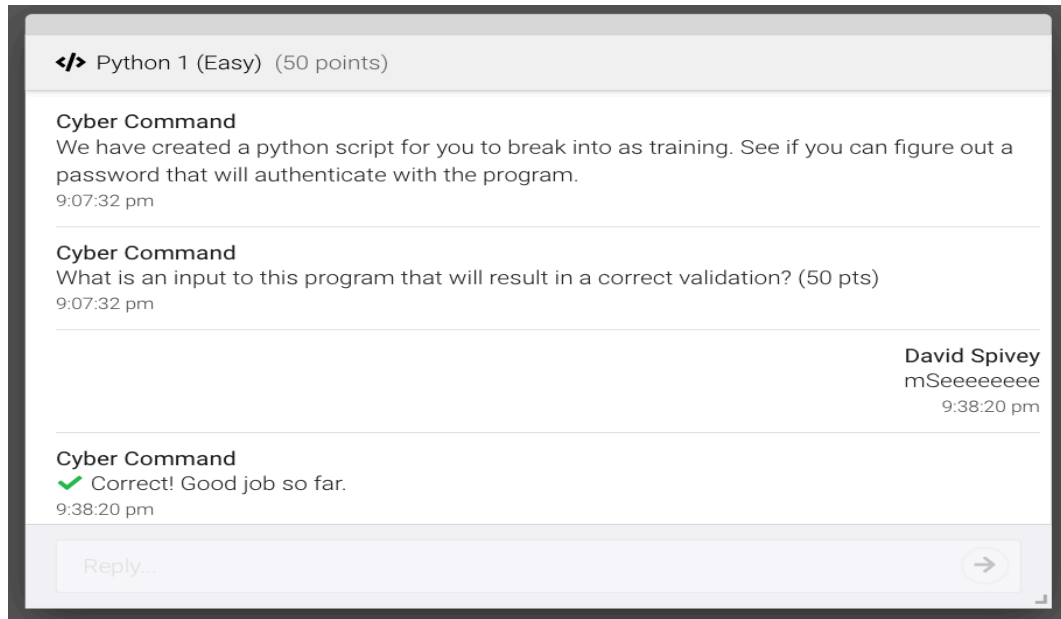
Completed by: David Spivey

Lab name: Python 1 (Easy)

Method of application: Python and Kali Linux

Procedure: To solve this I had to find a string of length 10 whose ASCII values sum to 1000 and has an s (ASCII code 83) as the second character in the string. The values of the other nine characters in the string are any combination of characters whose ASCII values sum to 917.

Flag proof: mSeeeeeeeee



Completed by: Jennifer Lorenzo

Lab name: Timebomb

Method of application: Linux terminal, JD-GUI

Procedure: The JD-GUI was used in order to decompile the Java file through a linux machine in order to successfully answer the three questions below.

Q1 - What programming language is the program written in? Java

```
public class Timebomb {
    public static void main(String[] var0) {
        double var1 = (double)System.currentTimeMillis();
        if (var1 > 1.65806352E12D) {
            System.out.println("Boom");
            System.exit(0);
        } else {
            System.out.println("It isn't time yet");
            System.exit(10);
        }
    }
}
```

Q2 - What exit code does the program exit with when it is not time? 10

```

public class Timebomb {
    public static void main(String[] var0) {
        double var1 = (double)System.currentTimeMillis();
        if (var1 > 1.65806352E12D) {
            System.out.println("Boom");
            System.exit(0);
        } else {
            System.out.println("It isn't time yet");
            System.exit(10);
        }
    }
}

```

Q3 - What time (in UTC) does the software go "Boom"? 2022-07-17 13:12

Convert milliseconds
1658063520000

to UTC time & date:
Sun Jul 17 2022 13:12:00

to local time & date:
Sun Jul 17 2022 09:12:00

Flag proof:

Timebomb (Easy) (45 points)

Success!
 You have completed all the questions for this challenge. Onto the next challenge! →

Cyber Command
 Reverse engineer a program to find out when it activates.
 10:07:01 pm

Q1 - 5 points	
What programming language is the program written in?	✓
Q2 - 10 points	
What exit code does the program exit with when it is not time?	✓
Q3 - 30 points	
What time (in UTC) does the software go "Boom"?	Activate Windows Go to Settings to activate Windows.

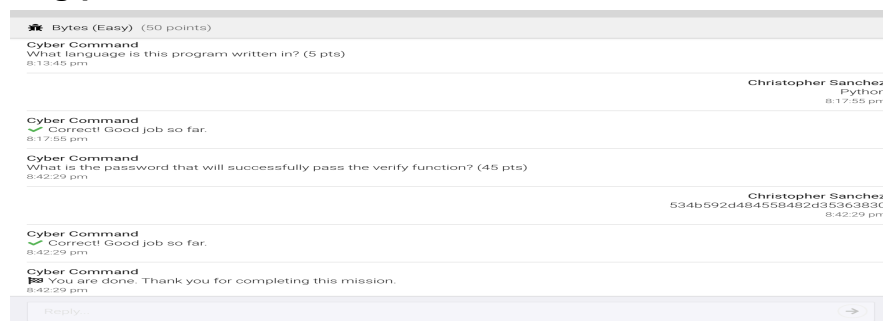
Completed by: Christopher Sanchez

Lab name: Bytes (Easy)

Method of application: Kali Linux

Procedure: To output the flag for this challenge we ensured to first attempt to run the python script in Kali Linux. We did this by saving the script into a .py file and running it in Kali Linux. Running the script would prompt us for a password so it was our job to reverse engineer the password of the script. This was done by researching a script online that would be able to convert the ASCII integers, provided by the script into hex form. Inputting the results of this into the password prompt outputted the flag.

Flag proof:



Completed by: Christopher Sanchez

Lab name: Binary 2 (Hard)

Method of application: Ghidra and Kali Linux

Procedure: To output the flag in this challenge, we had to use a combination of Kali Linux and Ghidra. Ghidra is used as our debugger in this example. First, to be able to analyze and inspect the binary file, we opened both binary files in Ghidra. Locating the 'Main' function in the Listing section of the tool, we find where that section in the debugger can be found in the decompiler section in Ghidra. We exploit the input function with a simple format vulnerability and like this we were able to output the flag.

Flag proof:

